LAB PROGRAM 8
ITERATIVE STATEMENTS – FOR LOOP

For Loop

A **for loop** is used to execute a block of code multiple times for each item in a sequence such as a **list, tuple, string, dictionary, set, or range of numbers**.

Syntax:

```
for variable in sequence:
     statements
```

- **variable** – takes the value of each item in the sequence
- **sequence** – collection of items (list, tuple, string, etc.)
- **statements** – code executed for every item

EXAMPLE:

```
fruits = ["apple", "banana", "cherry"]
for fru in fruits:
print("I like", fru)
```

OUTPUT

I like apple
I like banana
I like cherry

Using range() in a For Loop

The **range()** function generates a sequence of numbers.

Syntax:

```
for variable in range(start, stop, step):
```

- **start** – starting number (default 0)
- **stop** – ending number (not included)
- **step** – difference between numbers (default 1)

EXAMPLE:

```
for i in range(0,4):
    print(i)
```

OUTPUT:

```
0
1
2
3
```

ITERATING OVER DIFFERENT DATA TYPES

Data Type	Description	Example Code	Example Output

List	Ordered collection of items that can be changed.	`fruits = ["apple","banana","cherry"]`
		`for item in fruits:`
		`  print("I like",item)`
			I like Apple
			I like Banana
			I like cherry

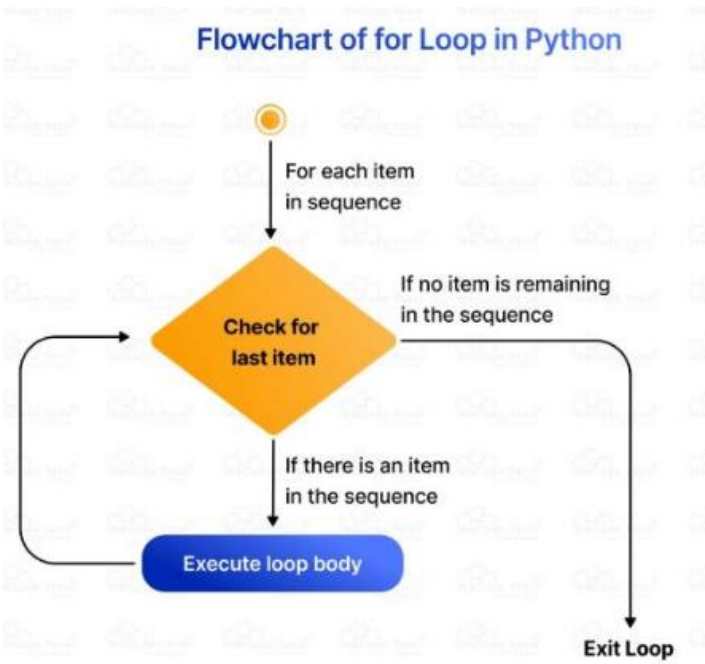
Tuple	Similar to list but immutable (cannot be changed).	`stationery = ("pen","pencil","eraser")`
		`for item in stationery:`
		`  print("this is my",item)`
			this is my Pen
			this is my Pencil
			this is my eraser

String	Sequence of characters. Loop accesses one character at a time.	`name = "Python"`
		`for char in name:`
		`  print(char)`
			P
			y
			t
			h
			o
			n

Dictionary	Stores data as key–value pairs. Loop iterates through keys.	`marks = {'Math':85,'English':90}`
		`for subject in marks:`
		`  print(subject, marks[subject])`
			Math 85
			English 90

Set	Unordered collection of unique elements.	`numbers = {10,20,30}`
		`for num in numbers:`
		`  print(num)`
			10
			20
			30

Using Index (range + len)	Access elements using their position (index).	`colors = ["red","green","blue"]`
		`for i in range(len(colors)):`
		`  print(colors[i])`
			red
			green
			blue



BREAK STATEMENTS:

- The break statement is used to terminate the loop prematurely, even if the loop condition is still true.
- When the break statement is encountered, the loop ends immediately, and the program control moves to the statement after the loop.

Syntax :

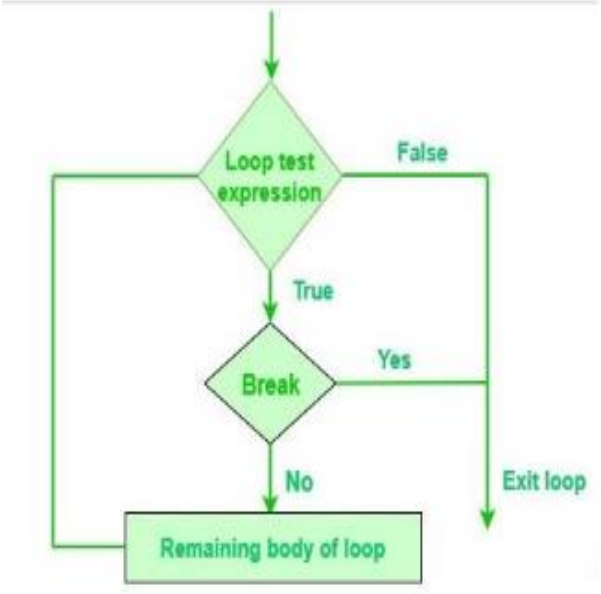
```
for variable in sequence:
    if condition:
        break
    # other statements
```

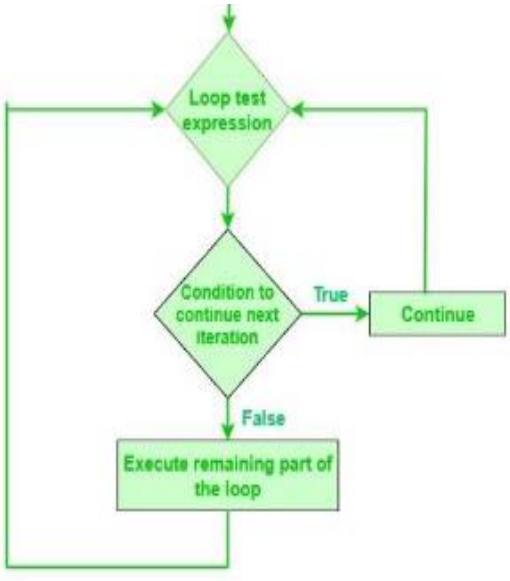
CONTINUE STATEMENT :

- The continue statement is used to skip the current iteration of the loop and jump directly to the next iteration.
- It does not terminate the entire loop; it just skips the remaining code in the current iteration.

Syntax :

```
for item in sequence:
    if condition:
        continue
          # other statements
```

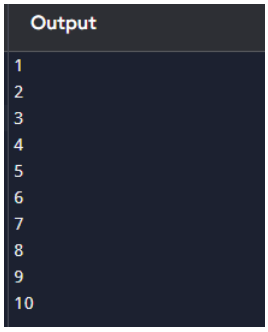




9A. Generate numbers from 1 to 10 using range().

PROGRAM:

```
for i in range(1, 11):
        print(i)
```



ALGORITHM:

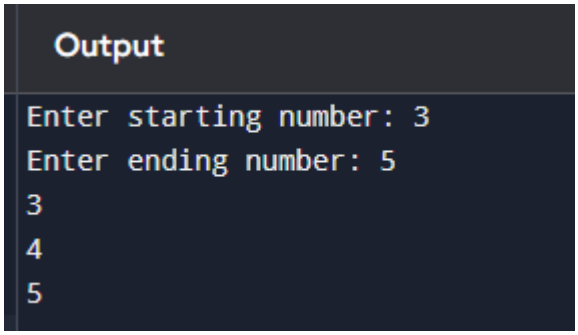
1. Start the program.
2. Use a **for loop** with `range(1,11)`.
3. The loop variable takes values from **1 to 10**.
4. Print the value of the loop variable in each iteration.
5. Repeat until the loop finishes.
6. Stop the program.

9B. Generate numbers for a given range.

PROGRAM:

```
start = int(input("Enter starting number: "))
end = int(input("Enter ending number: "))

for i in range(start, end + 1):
      print(i)
```



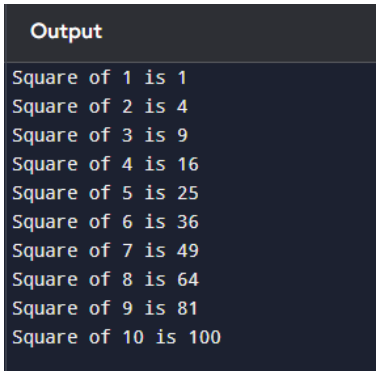
ALGORITHM:

1. Start the program.
2. Input the **starting number** from the user.
3. Input the **ending number** from the user.
4. Use a **for loop** with `range(start, end+1)`.
5. Print the value of the loop variable in each iteration.
6. Repeat until all numbers in the range are printed.
7. Stop the program.

9C. Write a program to display square numbers from 1 to 10.

PROGRAM:

```
for i in range(1, 11):
    print(f"Square of {i} is {i*i}")
```



ALGORITHM:

1. Start the program.
2. Initialize a **for loop** using `range(1, 11)`.
3. Take the value of `i` from the range (1 to 10).
4. Calculate the square of the number using `i * i`.
5. Display the square value.
6. Repeat steps 3–5 until all numbers from 1 to 10 are processed.
7. Stop the program.

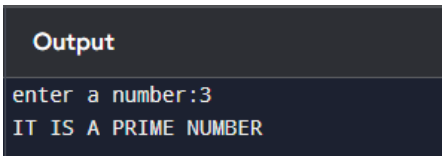
9D. Write a Python program that accepts an integer input from the user and determines whether the number is a prime number or not. If the number is prime, display an appropriate message; otherwise, indicate that it is not prime.

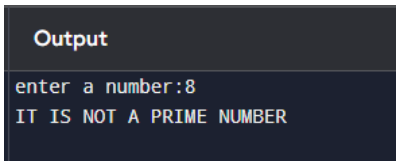
PROGRAM:

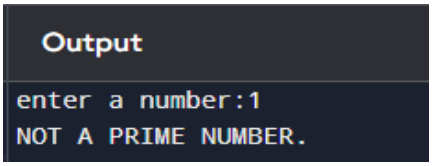
```
num = int(input("enter a number:"))

if num < 2:
    print("NOT A PRIME NUMBER.")
else:
    is_prime = True
    for i in range(2,num):
        if num % i == 0:
            is_prime = False
            break

    if is_prime:
        print("IT IS A PRIME NUMBER")
    else:
        print("IT IS NOT A PRIME NUMBER")
```







Algorithm :

1. Start the program.
2. Input a number **num** from the user.
3. Check if **num < 2**.
4. If true, print **"Not a prime number"**.
5. Otherwise, set a variable **is_prime = True**.
6. Use a **for loop** from **2 to num − 1**.
7. Check if **num % i == 0**.
8. If true, set **is_prime = False** and **break** the loop.
9. After the loop, check the value of **is_prime**.
10. If **True**, print **"It is a prime number"**.
11. Otherwise, print **"It is not a prime number"**.
12. Stop the program.

9E. Write a Python program that accepts a message and a number from the user. The program should then print the specified message the given number of times.

PROGRAM:

```
message = input("Enter the message: ")
n = int(input("Enter number of times: "))

for i in range(n):
    print(message)
```

```
Output
Enter the message: Hi Hello!!!
Enter number of times: 3
Hi Hello!!!
Hi Hello!!!
Hi Hello!!!
```

ALGORITHM:

1. Start the program.
2. Input a **message** from the user.
3. Input a **number (n)** from the user.
4. Start a **for loop** from 0 to n-1 using range(n).
5. Print the message.
6. Repeat steps 4–5 until the loop finishes.
7. Stop the program.

9F. Write a Python program to calculate the factorial of a given number. The program should:
a. Accept a non-negative integer as input.
b. Use an iterative approach to compute the factorial.
c. Handle invalid inputs (e.g., negative numbers or non-numeric inputs) by displaying appropriate error messages.

PROGRAM:

```
try:
    num = int(input("Enter a non-negative integer: "))

    if num < 0:
        print("Error: Factorial is not defined for negative numbers.")
    else:
        fact = 1
        for i in range(1, num + 1):
            fact = fact * i
        print(f"Factorial of {num} is =", fact)

except ValueError:
    print("Error: Please enter a valid integer.")
```

Output

```
Enter a non-negative integer: 5
Factorial of 5 is = 120
```

Output

```
Enter a non-negative integer: -9
ERROR!
Error: Factorial is not defined for negative numbers.
```

Output

```
Enter a non-negative integer: 2CA
ERROR!
Error: Please enter a valid integer.
```

ALGORITHM:

1. Start the program.
2. Ask the user to enter a **non-negative integer**.
3. Convert the input into an integer.
4. If the input is **not a valid integer**, display an error message.
5. Check if the number is **negative**.
6. If the number is negative, display **"Factorial is not defined for negative numbers."**
7. Otherwise, initialize **fact = 1**.
8. Use a **for loop** from **1 to num**.
9. Multiply **fact = fact × i** in each iteration.
10. After the loop finishes, display **the factorial of the number**.
11. Stop the program.

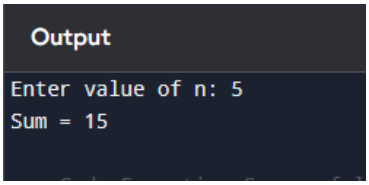
9G. Evaluation of mathematical series like

i. S=1+2+3+……+n

```
n = int(input("Enter value of n: "))
s = 0

for i in range(1, n+1):
    s = s + i

print("Sum =", s)
```



ALGORITHM:

- Start the program.
- Input the value of **n** from the user.
- Initialize **s = 0** (to store the sum).
- Start a **for loop** from **1 to n**.
- Add the current value of **i** to **s** (`s = s + i`).
- Repeat the loop until **i = n**.
- Display the value of **s** as the sum.
- Stop the program.

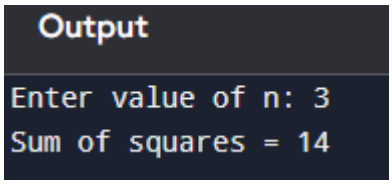
ii. $S = 1^2 + 2^2 + 3^2 + \cdots + n^2$

PROGRAM:

```
n = int(input("Enter value of n: "))
s = 0

for i in range(1, n+1):
    s = s + i*i

print("Sum of squares =", s)
```



ALGORITHM:

- Start
- Read the value of **n**
- Set **s = 0**
- For **i = 1 to n**
- Compute **i × i** (square of i)
- Add the value to **s**
- Repeat until **i = n**
- Print **s** (sum of squares)
- Stop

iii. $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$

```
n = int(input("Enter value of n: "))
s = 0

for i in range(1, n+1):
    s = s + 1/i

print("Harmonic Sum =", s)
```

```
Output
Enter value of n: 3
Harmonic Sum = 1.8333333333333333
```

ALGORITHM:

- Start
- Read the value of **n** from the user
- Initialize **s = 0**
- Start a loop from **i = 1 to n**
- Calculate **1 / i**
- Add the result to **s**
- Repeat steps 5–6 until **i = n**
- Display **s** (Harmonic Sum)
- Stop